DEPENDENCY SECURITY BEFORE EXECUTION

WARDEN

Every dependency change becomes a verified transaction.

Plan the complete graph. Approve only the authority required. Install with scripts suppressed. Prove what landed.


```
$ npm install esbuild
```

THE PROBLEM

The dangerous moment happens before a scanner can report it.

A lifecycle script runs with your permissions. By the time a post-install scan starts, credentials or source may already be gone.

01

request

one package name

02

resolve + execute

transitives and scripts

03

scan later

evidence after impact

Warden moves the decision here. Before download, unpack, or execution.

454,648 malicious packages found in 2025

205,474 unique fake package names generated in the study

THE HIDDEN CHANGE

You type one package. The manager installs a graph.

REQUESTED

esbuild

1 direct package

27 packages change

Checking only the name you typed leaves 26 transitive packages unexamined.

THE MODEL

Four transaction stages close four different failure modes.

01

PLAN

See the complete prospective graph and analyze every change.

prevents blind transitives

02

APPROVE

Name the exact script or coverage exception that needs authority.

prevents broad bypasses

03

APPLY

Replay the planned request with package scripts suppressed.

prevents pre-review execution

04

VERIFY

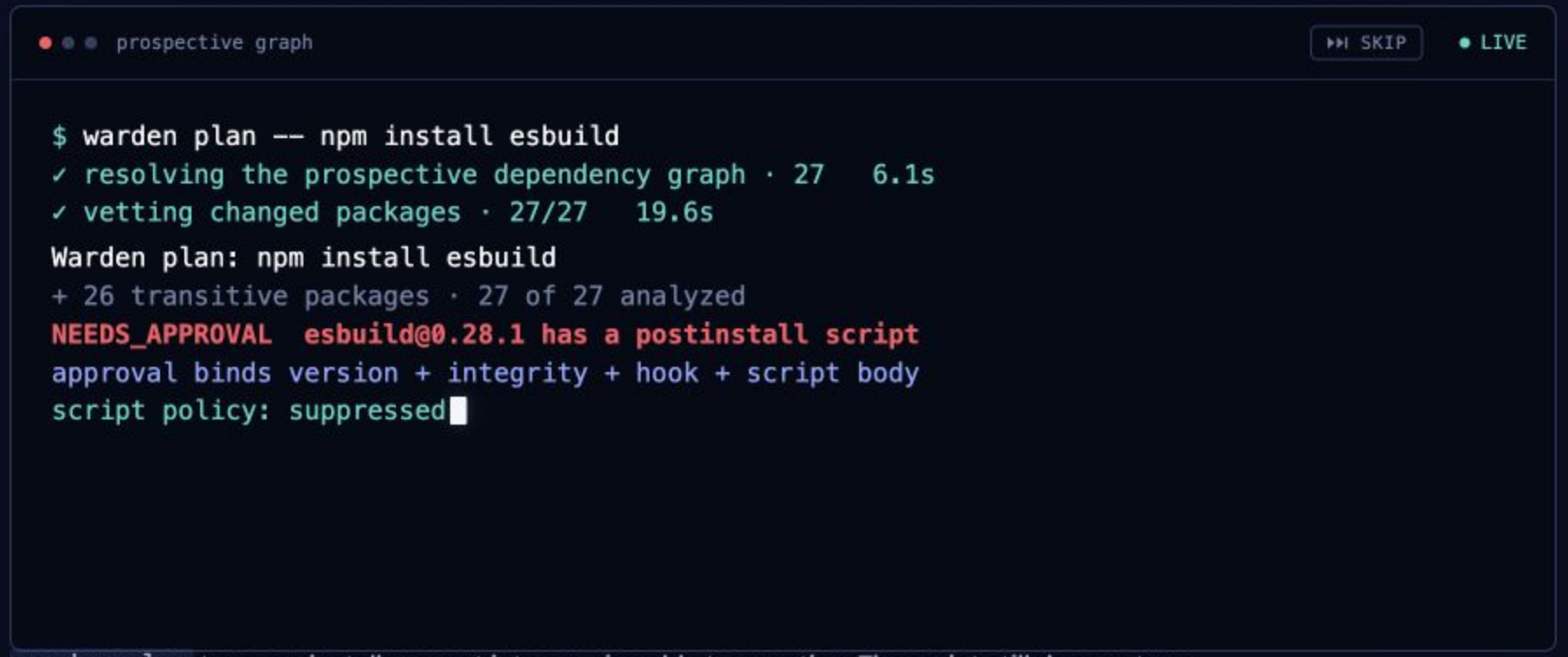
Compare the observed graph and project checks with the plan.

prevents silent drift

No complete analysis means no confident **ALLOW**.

PROBLEM 01 · BLIND GRAPH

A package name does not reveal the graph, scripts, or authority it will add.



REQUEST SHOWS

1 name

the direct package only

WARDEN PLAN REVEALS

27 changes

all 27 artifacts analyzed

SAFE NEXT STEP

1 approval

bound to the exact postinstall hook

`warden plan` turns an install request into a reviewable transaction. The script still does not run.

PROBLEM 02 · PLAN DRIFT

A safe plan is worthless if installation resolves something different.

REPLAY

exact planned request

manager, argv, working directory, dependency class

→

SUPPRESS

all lifecycle scripts

native manager setting recorded in the receipt

→

VERIFY

test, typecheck, build

stop at the first project failure

→

COMPARE

observed graph digest

must equal the graph that was reviewed

TRANSACTION RECEIPT

result: applied

```
graph reviewed = graph observed
script policy = suppressed
artifact coverage = complete
```

WHEN ANY CHECK FAILS

Restore the manifest and every lockfile.

Warden does not claim to restore node_modules or side effects from verification.

PROBLEM 03 · LOCAL BYPASS

A local guard can be bypassed. The pull request still needs proof.

GUIDANCE	**Teach the safe loop**	Instructions and agent setup make the intended path obvious.	can be ignored
INTERCEPTION	**Mediate package-manager commands**	Shims cover npm, pnpm, Yarn, Bun, npx, bunx, exec, dlx, ci, rebuilds, and global installs.	absolute paths and containers can bypass it
VERIFICATION	**Refuse the pull request without a valid receipt**	`warden ci --require-transaction-receipt`	does not trust the developer machine

dependency graph changed → **receipt verifies** → **MERGE**

PROBLEM 04 · REPUTATION

A trusted package name says nothing about this release's code or source.

PUBLISHED PACKAGE

resolve → integrity → release diff → AST scan → name intel → deterministic score

Finds known malware, name attacks, new execution, provenance changes, credential access, egress, destructive calls, and obfuscation combined with capability.

LOCKFILE

Where will bytes come from?

Off-registry hosts, plaintext transport, weak integrity, git and file sources.

SCRIPTS

What code runs during install?

Lifecycle hooks across the installed graph, including pipe-to-shell and credential access.

CONFIG

Where will credentials be sent?

Lookalike registries, plaintext tokens, and disabled TLS without echoing secret values.

`warden check` is deterministic. Verdicts are cached by tarball integrity and analyzer version, so repeat checks stay fast without reusing stale analysis.

PROBLEM 05 · MANAGER DRIFT

One safety policy becomes four opportunities for settings to diverge.

REPOSITORY INTENT

approved scripts
minimum release age
block exotic sources
reverify lockfiles
block downgrades

→

npm	ignore-scripts, min-release-age, source controls
pnpm	strict builds, minimum age, exotic-subdependency controls
Yarn	script control, hardened mode, age gate
Bun	install script suppression

HONEST GAP REPORTING

If the manager cannot express a rule, Warden says so and enforces it in the plan and receipt gate.

The invoked manager, packageManager field, lockfile, project config, and PATH decide which tool Warden preserves.

PROBLEM 06 · RISKY REMEDIATION

A vulnerability fix is not safe until the candidate release passes the gate.

01 **Find advisories**
Audit direct dependencies through OSV.

02 **Build repair plans**
Compare minimal and latest candidates.

03 **Gate every fix**
Reject the official fix if its release is risky.

04 **Verify in isolation**
Install without scripts, run project checks, then apply.

PROBLEM 07 · SEMANTIC FAILURE

Code can compile and still fail the request.

prompt versus diff

⏭ SKIP ● LIVE

```
$ warden intent check
prompt  "add rate limiting, keep retries, log every limited request"
3 claims extracted · 2 files changed
✓ delivered   add rate limiting
✓ preserved   retry logic untouched
✗ dropped     log every limited request
! scope creep  pagination.ts was never requested
! hallucinated  axios.instance.throttle is not exported
BLOCK  diff does not match the prompt · exit 20
```

When `.warden/prompt.txt` exists, CI runs the same intent check automatically for changed JavaScript and TypeScript.

DROPPED

A requested behavior never appeared.

SCOPE CREEP

Unrequested files or behavior changed.

HALLUCINATED API

Added code calls an export that does not exist.

PROBLEM 08 · UNSTRUCTURED AUTOMATION

Automation needs a decision contract, not terminal prose.

INPUT

package request or code diff

→

WARDEN

deterministic verdict

→

OUTPUT

fix + verify command

Stable contract

JSON output, published schemas, typed errors, and exit codes 0, 10, 20, 30.

Safe data boundary

Registry-authored text is sanitized and quarantined as untrusted input.

Capability-aware setup

Adapter setup reports which guidance, interception, post-change, and tool layers are actually available.

Restricted tools

The generated tool manifest exposes read-only decisions and excludes project-changing commands.

`warden handoff` writes the failing finding, evidence, concrete fix, and `warden ci --reporter agent` finish line

WHEN SOMETHING FAILS

Start with the operational problem. The command follows.

FAILURE	WARDEN RESPONSE	PROOF PRODUCED
RISKY RELEASE **The team needs evidence and a safer candidate.**	`check` `explain` `history` `compare` `baseline` `scripts`	**release verdict** evidence trail, candidate ranking, and exact remediation
BROKEN GUARDRAIL **Interception or repository setup may be incomplete.**	`coverage` `integrations doctor` `detect` `init` `config` `log`	**coverage and repair** missing layers, concrete fixes, and recorded verdict history
MISSING SYSTEM PROOF **CI and tools need stable, machine-readable evidence.**	`install` `ci` `schema` `completions` `benchmark` `uninstall`	**repeatable operations** manager-preserving execution, report contracts, and lifecycle control

MEASURED AND BOUNDED

The evidence is reproducible. The limits are explicit.

12 / 12

curated attack shapes stopped

0 / 9

benign shapes stopped

METHOD

Every case runs through the real resolver and transaction decision with complete graph coverage.

Regression corpus, not field accuracy.

Not a sandbox

Absolute paths and containers can bypass local shims.

Scripts stay suppressed

Packages that require install hooks may fail verification.

Rollback is scoped

Manifest and lockfiles return. node_modules does not.

Receipts are unsigned

Evidence, not independent attestation.

Technical alpha: a working control plane with honest boundaries.

THE OUTCOME

Keep the workflow. Add a chain of custody.

ONE DECISION CONTRACT

ALLOW · WARN · NEEDS_APPROVAL · BLOCK

local, package managers, automation, and CI

EVIDENCE

Sources

01 Sonatype software supply-chain report

02 USENIX package hallucination study

03 OSV advisory database

04 Warden benchmark corpus and method

05 Transaction model and limitations

06 Command interception coverage

Product claims correspond to the shipped command registry and repository documentation on this revision.